

a. Best Programming Practice

1. All values as variables including Fixed, User Inputs, and Results
2. Proper naming conventions for all variables
3. Proper Program Name and Class Name
4. Proper Method Name which indicates action taking inputs and providing result

Sample Program 1: Create a program to find the sum of all the digits of a number given by a user using an array and display the sum.

- b. Use Math.random() and get a 4-digit random integer number
- c. Write a method to count digits in the number
- d. Write a method to return an array of digits from a given number.
- e. Write a method to Find the sum of the digits of the number in the array
- f. Finally, display the sum of the digits of the number

Java

```
// Create SumOfDigit Class to compute the sum of 4 digits random number
class SumOfDigits {
    // Get a 4 digit random number
    public int get4DigitRandomNumber() {
        return (int) (Math.random() * 9000) + 1000;
    }

    // Find the count of digits in the number
    public int countDigits(int number) {
        int count = 0, temp = number;
        while (temp > 0) {
            count++;
            temp /= 10;
        }
        return count;
    }

    // Store the digits of the number in an array
    public int[] getDigits(int number, int count) {
        int[] digits = new int[count];
        int temp = number;
        for (int i = count - 1; i >= 0; i--) {
            digits[i] = temp % 10;
            temp /= 10;
        }
        return digits;
    }
}

// Find the sum of the elements in an array
```

```

public int sumArray(int[] array) {
    int sum = 0;
    for (int i = 0; i < array.length; i++) {
        sum += array[i];
    }
    return sum;
}

public static void main(String[] args) {
    // Get 4 digit random integer number
    SumOfDigits sumOfDigits = new SumOfDigits();
    int number = sumOfDigits.get4DigitRandomNumber();
    System.out.println("The Random Number is: " + number);

    // Get the count of digits
    int count = sumOfDigits.countDigits(number);
    System.out.println("The count of digit is: " + count);

    // Get the array of digits from the number
    int[] digits = sumOfDigits.getDigits(number, count);

    // Find the sum of the digits of the number
    int sum = sumOfDigits.sumArray(digits);

    // Display the sum of the digits of the number
    System.out.println("\nSum of Digits: " + sum);
}
}

```

Level 3 Practice Programs

1. Create a program to find the shortest, tallest, and mean height of players present in a football team.

Hint =>

- a. The formula to calculate the mean is: $\text{mean} = \text{sum of all elements} / \text{number of elements}$
- b. Create an int array named heights of size 11 and get 3 digits random height in cms for each player in the range 150 cms to 250 cms
- c. Write the method to Find the sum of all the elements present in the array.
- d. Write the method to find the mean height of the players on the football team
- e. Write the method to find the shortest height of the players on the football team
- f. Write the method to find the tallest height of the players on the football team
- g. Finally display the results

```
import java.util.Random;

public class FootballTeamHeights {
    public static void main(String[] args) {
        int[] heights = new int[11];
        Random random = new Random();

        // Generating random heights between 150 cm and 250 cm for each
        player
        for (int i = 0; i < heights.length; i++) {
            heights[i] = random.nextInt(101) + 150; // 150 + (0 to 100)
        }

        // Displaying heights of players
        System.out.print("Heights of players (in cms): ");
        for (int height : heights) {
            System.out.print(height + " ");
        }
        System.out.println();

        // Calculating and display the results
        int sum = calculateSum(heights);
        double mean = calculateMean(heights);
        int shortest = findShortestHeight(heights);
        int tallest = findTallestHeight(heights);

        System.out.println("Sum of heights: " + sum + " cms");
    }
}
```

```

        System.out.println("Mean height: " + mean + " cms");
        System.out.println("Shortest height: " + shortest + " cms");
        System.out.println("Tallest height: " + tallest + " cms");
    }

    // Method to calculate the sum of all elements in the array
    public static int calculateSum(int[] array) {
        int sum = 0;
        for (int value : array) {
            sum += value;
        }
        return sum;
    }

    // Method to calculate the mean height of players
    public static double calculateMean(int[] array) {
        int sum = calculateSum(array);
        return (double) sum / array.length;
    }

    // Method to find the shortest height of players
    public static int findShortestHeight(int[] array) {
        int shortest = array[0];
        for (int value : array) {
            if (value < shortest) {
                shortest = value;
            }
        }
        return shortest;
    }

    // Method to find the tallest height of players
    public static int findTallestHeight(int[] array) {
        int tallest = array[0];
        for (int value : array) {
            if (value > tallest) {
                tallest = value;
            }
        }
        return tallest;
    }
}

```

```
}
```

2. Extend or Create a **NumberChecker** utility class and perform following task. Call from main() method the different methods and display results. Make sure all are static methods

Hint =>

- a. Method to Find the count of digits in the number
- b. Method to Store the digits of the number in a digits array
- c. Method to Check if a number is a duck number using the digits array. A duck number is a number that has a non-zero digit present in it
- d. Method to check if the number is a armstrong number using the digits array. Armstrong number is a number that is equal to the sum of its own digits raised to the power of the number of digits. Eg: $153 = 1^3 + 5^3 + 3^3$
- e. Method to find the largest and second largest elements in the digits array. Use **Integer.MIN_VALUE** to initialize the variable.
- f. Method to find the the smallest and second smallest elements in the digits array. Use **Integer.MAX_VALUE** to initialize the variable.

```
import java.util.Scanner;
import java.lang.Math;
public class NumberChecker2 {

    //method to count digits of a number
    public static int digitCounter(int number) {

        //putting the value of number in temp variable
        int temp = number;

        //creating a variable count to count the digits
        int count = 0;

        //counting the digits
        while(temp != 0) {
            //dividing the number by 10
            temp /= 10;

            //incrementing the count
            count++;
        }
        //returning the count
    }
}
```

```

        return count;
    }

    //method to store digit in array
    public static int[] digitOfNumber(int number) {

        //putting the value of number in temp variable
        int temp = number;

        //creating a variable index to point index of array
        int index = 0;

        //creating the array digits to store digits of number
        int maxSize = 10;
        int[] digits = new int[maxSize];

        //storing the digits in digits array
        while(temp != 0) {

            //increasing the size of digits array if max size reached
            if(index == maxSize) {
                maxSize += 10;

                //creating tempARR array
                int[] tempARR = new int[maxSize];

                //copying the digits array into tempARR array
                for(int i=0; i<index+1; i++) {
                    tempARR[i] = digits[i];
                }
                digits = tempARR;
            }

            //storing the last digit in array
            digits[index] = temp%10;

            //dividing the number by 10
            temp /= 10;

            //incrementing the index

```

```

        index++;
    }

    //trimming the extra spaces
    int[] tempARR = new int[index];
    for(int i=0; i<index; i++) {
        tempARR[i] = digits[i];
    }
    digits = tempARR;

    //reversing the digits
    digits = reverseArray(digits);

    //returning the digits
    return digits;
}

//method to reverse an array
public static int[] reverseArray(int[] array) {

    //creating array reversedArray array to store reversed array
    int[] reversedArray = new int[array.length];

    //reversing the array
    for(int i=0; i<array.length; i++) {
        reversedArray[i] = array[array.length-i-1];
    }

    //returning the reversedArray
    return reversedArray;
}

//method to check whether two arrays are palindrome or not
public static boolean isPalindrome(int number) {

    //creating an array digits to store digits of number
    int digits[] = digitOfNumber(number);

    //creating an array reversedDigits to store reversed digits of
number
    int[] reversedDigits = reverseArray(digits);

```

```
//comparing each element of arrays
for(int i=0; i<(digits.length); i++) {

    //returning false if both values are different
    if(digits[i] != reversedDigits[i]) {
        return false;
    }
}
//returning true because all elements are same
return true;
}

//method to check whether number is duck number or not
public static boolean isDuckNumber(int number) {

    //creating an array digits to store digits of number
    int digits[] = digitOfNumber(number);

    //iterating through each element of array
    for(int i=0; i<(digits.length); i++) {

        //returning false if element is zero
        if(digits[i] == 0) {
            return false;
        }
    }
    //returning true because all elements are non-zero
    return true;
}

//method to check whether number is armstrong or not
public static boolean isArmStrong(int number) {

    //creating an array digits to store digits of number
    int digits[] = digitOfNumber(number);

    //creating a variable arm to store the sum of cubes of digits
    int arm = 0;
    //iterating through each element of array
    for(int i=0; i<(digits.length); i++) {
```

```

        arm += Math.pow(digits[i],3);

    }

    //returning true if number is armstrong else false
    if (arm == number) {
        return true;
    } else {
        return false;
    }
}

//method to find smallest and second smallest digits
public static int[] smallestAndSecondSmallestDigits(int number) {

    //creating an array digits to store digits of number
    int digits[] = digitOfNumber(number);

    int smallest = Integer.MAX_VALUE;
    int secondSmallest = Integer.MAX_VALUE;

    //finding smallest and second smallest digits
    for (int value : digits) {
        if (value < smallest) {
            secondSmallest = smallest;
            smallest = value;
        } else if (value < secondSmallest && value != smallest) {
            secondSmallest = value;
        }
    }

    //creating array to store smallest and second Smallest values
    int[] smallestAndSecondSmallest = {smallest, secondSmallest};

    //returning the smallest and second Smallest values
    return smallestAndSecondSmallest;
}

//method to find largest and second largest digits
public static int[] largestAndSecondLargestDigits(int number) {

```

```
//creating an array digits to store digits of number
int digits[] = digitOfNumber(number);

int largest = Integer.MIN_VALUE;
int secondLargest =Integer.MIN_VALUE;

//finding largest and second largest digit
for(int value: digits) {
    if(value > largest) {
        secondLargest = largest;
        largest = value;
    } else if (value > secondLargest && value != largest) {
        secondLargest = value;
    }
}

//creating array to store largest and second largest values
int[] largestAndSecondLargest = {largest, secondLargest};

//returning the largest and second largest values
return largestAndSecondLargest;
}

public static void main(String []args) {

    //creating scanner object to take user input
    Scanner input = new Scanner(System.in);

    //creating variable number to indicate the number
    int number;

    //taking user input for number
    System.out.print("Enter a number : ");
    number = input.nextInt();

    //creating variables count to indicate number of digits
    int count = digitCounter(number);

    //creating array digits to store digits of number
    int[] digits = digitOfNumber(number);
```

```

        //creating array reversedDigits to store digits in reversed
form
        int[] reversedDigits = reverseArray(digits);

        //creating the variables largest, secondLargest, smallest and
secondSmallest to storing the largest digit, second largest digit, smallest
digit and second smallest digit
        int largest = largestAndSecondLargestDigits(number)[0];
        int secondLargest = largestAndSecondLargestDigits(number)[1];
        int smallest = smallestAndSecondSmallestDigits(number)[0];
        int secondSmallest =
smallestAndSecondSmallestDigits(number)[1];

        //printing the details
        System.out.println("You Entered " + number);

        //printing the count of digits
        System.out.println("Number of digits is : " + count);

        //printing the digits of number
        System.out.print("digits are :");
        for(int i=0; i<digits.length; i++) {
            System.out.print(" " + digits[i]);
        }

        //printing the digits of number in reversed form
        System.out.print("\nReversed digits are :");
        for(int i=0; i<digits.length; i++) {
            System.out.print(" " + reversedDigits[i]);
        }

        //printing whether the number is palindrome or not
        if(isPalindrome(number)) {
            System.out.println("\nIt is a palindrome number.");
        } else {
            System.out.println("\nIt is not a palindrome number.");
        }

        //printing whether the number is duck number or not
        if(isDuckNumber(number)) {
            System.out.println("It is a duck number.");
        }
    }
}

```

```

    } else {
        System.out.println("It is not a duck number.");
    }
    //printing whether the number is armstrong number or not
    if(isArmStrong(number)) {
        System.out.println("It is a armstrong number.");
    } else {
        System.out.println("It is not a armstrong number.");
    }
    //printing the largest and second largest digits
    if (secondLargest == Integer.MIN_VALUE) {
        System.out.println("Largest digit: " + largest);
        System.out.println("No second largest digit (all elements might
be the same).");
    } else {
        System.out.println("Largest digit: " + largest);
        System.out.println("Second largest digit: " + secondLargest);
    }

    //printing the largest and second largest digits
    if (secondSmallest == Integer.MAX_VALUE) {
        System.out.println("Smallest digit: " + smallest);
        System.out.println("No second smallest digit (all elements
might be the same).");
    } else {
        System.out.println("Smallest digit: " + smallest);
        System.out.println("Second smallest digit: " + secondSmallest);
    }

    //closing the input object
    input.close();
}
}

```

3. Extend or Create a **NumberChecker** utility class and perform following task. Call from main() method the different methods and display results. Make sure all are static methods

Hint =>

- a. Method to find the count of digits in the number and a Method to Store the digits of the number in a digits array

- b. Method to find the sum of the digits of a number using the digits array
- c. Method to find the sum of the squares of the digits of a number using the digits array. Use ***Math.pow()*** method
- d. Method to Check if a number is a harshad number using a digits array. A number is called a Harshad number if it is divisible by the sum of its digits. For e.g. 21
- e. Method to find the frequency of each digit in the number. Create a 2D array to store the frequency with digit in the first column and frequency in the second column.

```
import java.util.Scanner;
import java.util.Arrays;
public class NumberChecker3 {

    //method to count digits of a number
    public static int digitCounter(int number) {

        //putting the value of number in temp variable
        int temp = number;

        //creating a variable count to count the digits
        int count = 0;

        //counting the digits
        while(temp != 0) {
            //dividing the number by 10
            temp /= 10;

            //incrementing the count
            count++;
        }
        //returning the count
        return count;
    }

    //method to store digit in array
    public static int[] digitOfNumber(int number) {

        //putting the value of number in temp variable
        int temp = number;

        //creating a variable index to point index of array
        int index = 0;
```

```
//creating the array digits to store digits of number
int maxSize = 10;
int[] digits = new int[maxSize];

//storing the digits in digits array
while(temp != 0) {

    //increasing the size of digits array if max size reached
    if(index == maxSize) {
        maxSize += 10;

        //creating tempARR array
        int[] tempARR = new int[maxSize];

        //copying the digits array into tempARR array
        for(int i=0; i<index+1; i++) {
            tempARR[i] = digits[i];
        }
        digits = tempARR;
    }
    //storing the last digit in array
    digits[index] = temp%10;

    //dividing the number by 10
    temp /= 10;

    //incrementing the index
    index++;
}

//trimming the extra spaces
int[] tempARR = new int[index];
for(int i=0; i<index; i++) {
    tempARR[i] = digits[i];
}
digits = tempARR;

//reversing the digits
digits = reverseArray(digits);
```

```

        //returning the digits
        return digits;
    }

    //method to reverse an array
    public static int[] reverseArray(int[] array) {

        //creating array reversedArray array to store reversed array
        int[] reversedArray = new int[array.length];

        //reversing the array
        for(int i=0; i<array.length; i++) {
            reversedArray[i] = array[array.length-i-1];
        }

        //returning the reversedArray
        return reversedArray;
    }

    //method to find sum of digits
    public static int sumOfDigits(int number) {

        //creating an array digits to store digits of number
        int digits[] = digitOfNumber(number);

        //creating variable sum to indicate the sum of digits
        int sum = 0;

        //calculating sum of digits
        for(int i=0; i<(digits.length); i++) {
            sum += digits[i];
        }
        //returning the sum
        return sum;
    }

    //method to find sum of squares of digits
    public static int sumOfSquareOfDigits(int number) {

        //creating an array digits to store digits of number
    
```

```

        int digits[] = digitOfNumber(number);

        //creating variable sumOfSq to indicate the sum squares of
digits
        int sumOfSq = 0;

        //calculating sum of square of digits
        for(int i=0; i<(digits.length); i++) {
            sumOfSq += digits[i] * digits[i];
        }
        //returning the sum of square of digits
        return sumOfSq;
    }

    //method to check whether number is harshad number or not
    public static boolean isHarshad(int number) {

        //calculating sum of digits
        int sumOfDigits = sumOfDigits(number);

        //checking number is harshad number or not
        if(number%sumOfDigits == 0) {
            //returning true because number is divisible by sum of
digit of number
            return true;
        }

        //returning false because number is not divisible by sum of
digit of number
        return false;
    }

    //method to find frequency of digits in a number
    public static int[][] frequency(int number) {

        //creating an array digits to store digits of number
        int digits[] = digitOfNumber(number);

        //sorting the array
        Arrays.sort(digits);
    }

```

```
//creating a 2d array tempFrequencyArray to store frequency of
digits in the form of {digit, frequency}
int[][] tempFrequencyArray = new int[digits.length][2];

// Starting with the first number
int currentNumber = digits[0];
int count = 0, index = 0;

// Counting frequencies
for (int num : digits) {
    if (num == currentNumber) {
        count++;
    } else {
        // Storing the number
        tempFrequencyArray[index][0] = currentNumber;
        // Storing its frequency
        tempFrequencyArray[index][1] = count;
        index++;
        // Updating to the next number
        currentNumber = num;
        count = 1;
    }
}

// Storing the last number and its frequency
tempFrequencyArray[index][0] = currentNumber;
tempFrequencyArray[index][1] = count;
index++;

// Creating the final 2D array with the exact size
int[][] frequencyArray = new int[index][2];
for (int i = 0; i < index; i++) {
    frequencyArray[i][0] = tempFrequencyArray[i][0];
    frequencyArray[i][1] = tempFrequencyArray[i][1];
}

//returning the frequencies
return frequencyArray;
}

public static void main(String []args) {
```

```
//creating scanner object to take user input
Scanner input = new Scanner(System.in);

//creating variable number to indicate the number
int number;

//taking user input for number
System.out.print("Enter a number : ");
number = input.nextInt();

//creating variables count to indicate number of digits
int count = digitCounter(number);

//creating variable sumOfDigits to store sum of digits
int sumOfDigits = sumOfDigits(number);

//creating variable sumOfDigits to store sum of squares of
digits
int sumOfSq = sumOfSquareOfDigits(number);

//creating array digits to store digits of number
int[] digits = digitOfNumber(number);

//creating a 2d array to store frequencies of number
int frequencyArray[][] = frequency(number);

//printing the details
System.out.println("You Entered " + number);

//printing the count of digits
System.out.println("Number of digits is : " + count);

//printing the digits of number
System.out.print("digits are : ");
for(int i=0; i<digits.length; i++) {
    System.out.print(" " + digits[i]);
}
//printing the sum of digits
System.out.println("\nSum of digits is : " + sumOfDigits);
```

```
//printing the sum of square of digits
System.out.println("Sum of square of digits is : " + sumOfSq);

//printing whether the number is harshad or not
if(isHarshad(number)) {
    System.out.println("It is a harshad number.");
} else {
    System.out.println("It is not a harshad number.");
}
//printing frequencies of each numbers
System.out.println("Number\tFrequency");
for (int[] row : frequencyArray) {
    System.out.println(row[0] + "\t" + row[1]);
}
//closing the input object
input.close();
}
}
```

4. Extend or Create a **NumberChecker** utility class and perform following task. Call from main() method the different methods and display results. Make sure all are static methods

Hint =>

- Method to find the count of digits in the number and a Method to Store the digits of the number in a digits array
- Method to reverse the digits array
- Method to compare two arrays and check if they are equal
- Method to check if a number is a palindrome using the Digits. A palindrome number is a number that remains the same when its digits are reversed.
- Method to Check if a number is a duck number using the digits array. A duck number is a number that has a non-zero digit present in it

```
import java.util.Scanner;
public class NumberChecker {

    //method to count digits of a number
    public static int digitCounter(int number) {

        //putting the value of number in temp variable
        int temp = number;
```

```
//creating a variable count to count the digits
int count = 0;

//counting the digits
while(temp != 0) {
    //dividing the number by 10
    temp /= 10;

    //incrementing the count
    count++;
}
//returning the count
return count;
}

//method to store digit in array
public static int[] digitOfNumber(int number) {

    //putting the value of number in temp variable
    int temp = number;

    //creating a variable index to point index of array
    int index = 0;

    //creating the array digits to store digits of number
    int maxSize = 10;
    int[] digits = new int[maxSize];

    //storing the digits in digits array
    while(temp != 0) {

        //increasing the size of digits array if max size reached
        if(index == maxSize) {
            maxSize += 10;

            //creating tempARR array
            int[] tempARR = new int[maxSize];

            //copying the digits array into tempARR array
```

```

        for(int i=0; i<index+1; i++) {
            tempARR[i] = digits[i];
        }
        digits = tempARR;

    }
    //storing the last digit in array
    digits[index] = temp%10;

    //dividing the number by 10
    temp /= 10;

    //incrementing the index
    index++;
}

//trimming the extra spaces
int[] tempARR = new int[index];
for(int i=0; i<index; i++) {
    tempARR[i] = digits[i];
}
digits = tempARR;

//reversing the digits
digits = reverseArray(digits);

//returning the digits
return digits;
}

//method to reverse an array
public static int[] reverseArray(int[] array) {

    //creating array reversedArray array to store reversed array
    int[] reversedArray = new int[array.length];

    //reversing the array
    for(int i=0; i<array.length; i++) {
        reversedArray[i] = array[array.length-i-1];
    }
}

```

```

        //returning the reversedArray
        return reversedArray;
    }

    //method to check whether two arrays are palindrome or not
    public static boolean isPalindrome(int number) {

        //creating an array digits to store digits of number
        int digits[] = digitOfNumber(number);

        //creating an array reversedDigits to store reversed digits of
number
        int[] reversedDigits = reverseArray(digits);

        //comparing each element of arrays
        for(int i=0; i<(digits.length); i++) {

            //returning false if both values are different
            if(digits[i] != reversedDigits[i]) {
                return false;
            }
        }
        //returning true because all elements are same
        return true;
    }

    //method to check whether number is duck number or not
    public static boolean isDuckNumber(int number) {

        //creating an array digits to store digits of number
        int digits[] = digitOfNumber(number);

        //iterating through each element of array
        for(int i=0; i<(digits.length); i++) {

            //returning false if element is zero
            if(digits[i] == 0) {
                return false;
            }
        }
        //returning true because all elements are non-zero
    }

```

```

        return true;
    }

    public static void main(String []args) {

        //creating scanner object to take user input
        Scanner input = new Scanner(System.in);

        //creating variable number to indicate the number
        int number;

        //taking user input for number
        System.out.print("Enter a number : ");
        number = input.nextInt();

        //creating variables count to indicate number of digits
        int count = digitCounter(number);

        //creating array digits to store digits of number
        int[] digits = digitOfNumber(number);

        //creating array reversedDigits to store digits in reversed
        int[] reversedDigits = reverseArray(digits);

        //printing the details
        System.out.println("You Entered " + number);

        //printing the count of digits
        System.out.println("Number of digits is : " + count);

        //printing the digits of number
        System.out.print("digits are : " );
        for(int i=0; i<digits.length; i++) {
            System.out.print(" " + digits[i]);
        }

        //printing the digits of number in reversed form
        System.out.print("\nReversed digits are : " );
        for(int i=0; i<digits.length; i++) {
            System.out.print(" " + reversedDigits[i]);
        }
    }
}

```

```

    }

    //printing whether the number is palindrome or not
    if(isPalindrome(number)) {
        System.out.println("\nIt is a palindrome number.");
    } else {
        System.out.println("\nIt is not a palindrome number.");
    }

    //printing whether the number is duck number or not
    if(isDuckNumber(number)) {
        System.out.println("It is a duck number.");
    } else {
        System.out.println("It is not a duck number.");
    }

    //closing the input object
    input.close();
}
}

```

5. Extend or Create a **NumberChecker** utility class and perform following task. Call from main() method the different methods and display results. Make sure all are static methods

Hint =>

- Method to Check if a number is prime number. A prime number is a number greater than 1 that has no positive divisors other than 1 and itself.
- Method to Check if a number is a neon number. A neon number is a number where the sum of digits of the square of the number is equal to the number itself
- Method to Check if a number is a spy number. A number is called a spy number if the sum of its digits is equal to the product of its digits
- Method to Check if a number is an automorphic number. An automorphic number is a number whose square ends with the number itself. E.g. 5 is an automorphic number
- Method to Check if a number is a buzz number. A buzz number is a number that is either divisible by 7 or ends with 7

```

import java.util.Scanner;
public class NumberChecker4 {
    // Main method
    public static void main(String[] args) {

```

```
// Creating a scanner object to take user input
Scanner input = new Scanner(System.in);

// Take user input for the number
System.out.print("Enter a number: ");
int number = input.nextInt();
// Checking if the number is prime
boolean isPrime = isPrime(number);
// Checking if the number is a neon number
boolean isNeon = isNeon(number);
// Checking if the number is a spy number
boolean isSpy = isSpy(number);
// Checking if the number is an automorphic number
boolean isAutomorphic = isAutomorphic(number);
// Checking if the number is a buzz number
boolean isBuzz = isBuzz(number);
// Displaying the results
System.out.println("\nResults:");
System.out.println("Is Prime: " + isPrime);
System.out.println("Is Neon: " + isNeon);
System.out.println("Is Spy: " + isSpy);
System.out.println("Is Automorphic: " + isAutomorphic);
System.out.println("Is Buzz: " + isBuzz);
// Closing the Scanner object
input.close();
}

// Method to check if a number is a prime number
public static boolean isPrime(int number) {
    // A prime number is greater than 1
    if (number <= 1) {
        return false;
    }
    // Checking for factors other than 1 and itself
    for (int i = 2; i <= Math.sqrt(number); i++) {
        if (number % i == 0) {
            return false;
        }
    }
    return true;
}
```

```
// Method to check if a number is a neon number
public static boolean isNeon(int number) {

    // Calculating the square of the number
    int square = number * number;

    // Calculating the sum of digits of the square
    int sumOfDigits = 0;
    while (square > 0) {
        sumOfDigits += square % 10;
        square /= 10;
    }
    // Checking if the sum of digits is equal to the number
    return sumOfDigits == number;
}

// Method to check if a number is a spy number
public static boolean isSpy(int number) {

    // Calculating the sum and product of the digits
    int sumOfDigits = 0;
    int productOfDigits = 1;
    while (number > 0) {
        int digit = number % 10;
        sumOfDigits += digit;
        productOfDigits *= digit;
        number /= 10;
    }
    // Checking if the sum of digits is equal to the product of
digits
    return sumOfDigits == productOfDigits;
}

// Method to check if a number is an automorphic number
public static boolean isAutomorphic(int number) {

    // Calculating the square of the number
    int square = number * number;

    // Checking if the square ends with the number itself
    while (number > 0) {
        if (number % 10 != square % 10) {
            return false;
        }
        number /= 10;
        square /= 10;
    }
    return true;
}
```

```

        }
        number /= 10;
        square /= 10;
    }
    return true;
}
// Method to check if a number is a buzz number
public static boolean isBuzz(int number) {

    // Checking if the number is divisible by 7 or ends with 7
    return number % 7 == 0 || number % 10 == 7;
}
}

```

6. Extend or Create a **NumberChecker** utility class and perform following task. Call from main() method the different methods and display results. Make sure all are static methods

Hint =>

- Method to find factors of a number and return them as an array. Note there are 2 for loops one for the count and another for finding the factor and storing in the array
- Method to find the greatest factor of a Number using the factors array
- Method to find the sum of the factors using factors array and return the sum
- Method to find the product of the factors using factors array and return the product
- Method to find product of cube of the factors using the factors array. Use **Math.pow()**
- Method to Check if a number is a perfect number. Perfect numbers are positive integers that are equal to the sum of their proper divisors
- Method to find the number is a abundant number. A number is called an abundant number if the sum of its proper divisors is greater than the number itself
- Method to find the number is a deficient number. A number is called a deficient number if the sum of its proper divisors is less than the number itself
- Method to Check if a number is a strong number. A number is called a strong number if the sum of the factorial of its digits is equal to the number itself

```

import java.util.Scanner;
import java.util.Arrays;
public class NumberChecker5 {

    // Main method
    public static void main(String[] args) {

        // Creating a scanner object to take user input
        Scanner input = new Scanner(System.in);
    }
}

```

```
// Taking user input for the number
System.out.print("Enter a number: ");
int number = input.nextInt();

// Finding the factors of the number
int[] factors = findFactors(number);

// Finding the greatest factor of the number
int greatestFactor = findGreatestFactor(factors);

// Finding the sum of the factors
int sumFactors = sumOfFactors(factors);

// Finding the product of the factors
int productFactors = productOfFactors(factors);

// Finding the product of the cubes of the factors
double productCubesFactors = productOfCubesOfFactors(factors);

// Checking if the number is a perfect number
boolean isPerfect = isPerfectNumber(number);

// Checking if the number is an abundant number
boolean isAbundant = isAbundantNumber(number);

// Checking if the number is a deficient number
boolean isDeficient = isDeficientNumber(number);

// Checking if the number is a strong number
boolean isStrong = isStrongNumber(number);

// Displaying the results
System.out.println("\nResults:");
System.out.println("Factors: " + Arrays.toString(factors));
System.out.println("Greatest Factor: " + greatestFactor);
System.out.println("Sum of Factors: " + sumFactors);
System.out.println("Product of Factors: " + productFactors);
System.out.println("Product of Cubes of Factors: " +
productCubesFactors);
System.out.println("Is Perfect Number: " + isPerfect);
```

```

System.out.println("Is Abundant Number: " + isAbundant);
System.out.println("Is Deficient Number: " + isDeficient);
System.out.println("Is Strong Number: " + isStrong);

// Closing the Scanner object
input.close();
}
// Method to find factors of a number and return them as an array
public static int[] findFactors(int number) {
    int count = 0;
    // Counting the number of factors
    for (int i = 1; i <= number; i++) {
        if (number % i == 0) {
            count++;
        }
    }
    int[] factors = new int[count];
    int index = 0;

    // Storing the factors in the array
    for (int i = 1; i <= number; i++) {
        if (number % i == 0) {

            factors[index++] = i;
        }
    }
    return factors;
}
// Method to find the greatest factor of a number using the factors
array
public static int findGreatestFactor(int[] factors) {
    int greatest = factors[0];

    // Finding the greatest factor in the array
    for (int factor : factors) {
        if (factor > greatest) {
            greatest = factor;
        }
    }
    return greatest;
}
}

```

```
// Method to find the sum of the factors using the factors array
public static int sumOfFactors(int[] factors) {
    int sum = 0;
    //calculating Sum of factors in the array
    for (int factor : factors) {
        sum += factor;
    }
    return sum;
}

// Method to find the product of the factors using the factors array
public static int productOfFactors(int[] factors) {
    int product = 1;

    // Multiplying the factors in the array
    for (int factor : factors) {
        product *= factor;
    }
    return product;
}

// Method to find the product of the cubes of the factors using the
factors array
public static double productOfCubesOfFactors(int[] factors) {
    double product = 1;

    // Multiplying the cubes of the factors in the array
    for (int factor : factors) {
        product *= Math.pow(factor, 3);
    }
    return product;
}

// Method to check if a number is a perfect number
public static boolean isPerfectNumber(int number) {
    int[] factors = findFactors(number);

    // calculating Sum the factors excluding the number itself
    int sum = sumOfFactors(factors) - number;

    // Checking if the sum is equal to the number
    return sum == number;
}

// Method to check if a number is an abundant number
```

```

public static boolean isAbundantNumber(int number) {
    int[] factors = findFactors(number);

    // calculating Sum the factors excluding the number itself
    int sum = sumOfFactors(factors) - number;

    // Checking if the sum is greater than the number
    return sum > number;
}
// Method to check if a number is a deficient number
public static boolean isDeficientNumber(int number) {
    int[] factors = findFactors(number);

    //calculating Sum the factors excluding the number itself
    int sum = sumOfFactors(factors) - number;

    // Checking if the sum is less than the number
    return sum < number;
}
// Helper method to calculate factorial of a digit
public static int factorial(int digit) {
    int fact = 1;

    // Calculate the factorial of the digit
    for (int i = 1; i <= digit; i++) {

        fact *= i;
    }
    return fact;
}
// Method to check if a number is a strong number
public static boolean isStrongNumber(int number) {
    int sum = 0;
    int temp = number;
    // Calculating the sum of the factorials of the digits
    while (temp != 0) {
        int digit = temp % 10;
        sum += factorial(digit);
        temp /= 10;
    }
    // Checking if the sum is equal to the number

```

```
        return sum == number;
    }
}
```

7. Write a program to generate a six-digit OTP number using Math.random() method. Validate the numbers are unique by generating the OTP number 10 times and ensuring all the 10 OTPs are not the same

Hint =>

- Write a method to Generate a 6-digit OTP number using Math.random()
- Create an array to save the OTP numbers generated 10 times
- Write a method to ensure that the OTP numbers generated are unique. If unique return true else return false

```
import java.util.HashSet;
import java.util.Random;

public class OTPGenerator {
    public static void main(String[] args) {
        int[] otpArray = new int[10];

        // Generating 10 OTPs and store them in an array
        for (int i = 0; i < otpArray.length; i++) {
            otpArray[i] = generateOTP();
        }

        // Displaying the generated OTPs
        System.out.println("Generated OTPs:");
        for (int otp : otpArray) {
            System.out.println(otp);
        }

        // Checking if all OTPs are unique
        boolean areUnique = areOTPsUnique(otpArray);
        System.out.println("Are all OTPs unique? " + (areUnique ? "Yes" :
"No"));
    }

    // Method to generate a 6-digit OTP using Math.random()
    public static int generateOTP() {
```

```

        return (int) (Math.random() * 900000) + 100000; // Generates a
number between 100000 and 999999
    }

    // Method to check if all OTPs in the array are unique
    public static boolean areOTPsUnique(int[] otpArray) {
        HashSet<Integer> uniqueOTPs = new HashSet<>();
        for (int otp : otpArray) {
            if (!uniqueOTPs.add(otp)) {
                // If adding to the set fails, it means the OTP is not
unique
                return false;
            }
        }
        return true;
    }
}

```

8. Create a program to display a calendar for a given month and year. The program should take the month and year as input from the user and display the calendar for that month. E.g. for 07 2005 user input, the program should display the calendar as shown below

```

July 2005
Sun Mon Tue Wed Thu Fri Sat
          1  2
 3   4   5   6   7   8   9
10  11  12  13  14  15  16
17  18  19  20  21  22  23
24  25  26  27  28  29  30
31

```

Hint =>

- Write a Method to get the name of the month. For this define a month Array to store the names of the months
- Write a Method to get the number of days in the month. For this define a days Array to store the number of days in each month. For Feb month, check for Leap Year to get the number of days. Also, define a Leap Year Method.
- Write a method to get the first day of the month using the Gregorian calendar algorithm

$$y0 = y - (14 - m) / 12$$

$$x = y0 + y0/4 - y0/100 + y0/400$$

$$m0 = m + 12 \times ((14 - m) / 12) - 2$$

$$d0 = (d + x + 31m0 / 12) \bmod 7$$

- d. Displaying the Calendar requires 2 **for** loops.
- The first **for** loop up to the first day to get the proper indentation. As in the example above 3 spaces from Sun to Thu as to be set as July 1st starts on Fri
 - The Second **for** loop Displays the days of the month starting from 1 to the number of days. Add proper indentation for single-digit days using **%3d** to display the integer right-justified in a field of width 3. Please note to move to the next line after Sat

```
import java.util.Scanner;

public class CalendarDisplay {
    // Method to get the name of the month
    public static String getMonthName(int month) {
        String[] months = {"", "January", "February", "March", "April",
                           "May", "June", "July", "August", "September",
                           "October", "November", "December"};
        return months[month];
    }

    // Method to get the number of days in the month
    public static int getNumberOfDays(int month, int year) {
        int[] days = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

        // Checking for leap year condition in February
        if (month == 2 && (year % 4 == 0 && (year % 100 != 0 || year % 400
        == 0))) {
            return 29;
        } else {
            return days[month];
        }
    }

    // Method to get the first day of the month using the Gregorian
    calendar algorithm
    public static int getFirstDayOfMonth(int month, int year) {
        int d = 1; // Setting the first day of the month
        int y = year - (14 - month) / 12;
        int x = y + y / 4 - y / 100 + y / 400;
        int m = month + 12 * ((14 - month) / 12) - 2;
        return (d + x + (31 * m) / 12) % 7;
    }
}
```

```

}

// Method to display the calendar for a given month and year
public static void displayCalendar(int month, int year) {
    System.out.println(" " + getMonthName(month) + " " + year);
    System.out.println("Su Mo Tu We Th Fr Sa");

    // Getting the number of days in the month
    int numberOfDays = getNumberOfDays(month, year);
    // Getting the first day of the month
    int startDay = getFirstDayOfMonth(month, year);

    // Printing spaces for the days before the first day
    for (int i = 0; i < startDay; i++) {
        System.out.print("  "); // Each space accounts for a day
    }

    // Printing the days of the month
    for (int day = 1; day <= numberOfDays; day++) {
        System.out.printf("%3d", day);
        if ((day + startDay) % 7 == 0) {
            // Starting a new line after Saturday
            System.out.println();
        }
    }

    // Printing a final newline after the calendar
    System.out.println();
}

// Main method
public static void main(String[] args) {
    // Creating a scanner object to take user input
    Scanner input = new Scanner(System.in);

    // Taking user input for the month and year
    System.out.print("Enter the month (1-12): ");
    int month = input.nextInt();
    System.out.print("Enter the year: ");
    int year = input.nextInt();

    // Displaying the calendar for the given month and year

```

```
displayCalendar(month, year);

// Closing the scanner object
input.close();
}
}
```

9. Write a program Euclidean distance between two points as well as the equation of the line using those two points. Use Math functions **Math.pow()** and **Math.sqrt()**

Hint =>

- Take inputs for 2 points x1, y1, and x2, y2
- Method to find the Euclidean distance between two points and return the distance

$$distance = \sqrt{(x2 - x1)^2} + \sqrt{(y2 - y1)^2}$$

- Write a Method to find the equation of a line given two points and return the equation which includes the slope and the y-intercept

The equation of a line is given by the equation $y = m * x + b$ Where m is the slope and b is the y-intercept. So firstly compute the slope using the formulae

$$m = (y2 - y1)/(x2 - x1)$$

Post that compute the y-intercept b using the formulae

$$b = y1 - m * x1$$

Finally, return an array having slope m and y-intercept b

```
import java.util.Scanner;

public class PointsCalculator {
    // Main method
    public static void main(String[] args) {
        // Creating a scanner object to take user input
        Scanner input = new Scanner(System.in);

        // Taking user input for the coordinates of the two points
        System.out.print("Enter x1: ");
        double x1 = input.nextDouble();
        System.out.print("Enter y1: ");
        double y1 = input.nextDouble();
        System.out.print("Enter x2: ");
        double x2 = input.nextDouble();
        System.out.print("Enter y2: ");
        double y2 = input.nextDouble();
```

```
// Calculating the Euclidean distance
double distance = findEuclideanDistance(x1, y1, x2, y2);
System.out.println("Euclidean Distance: " + distance);

// Calculating the equation of the line
double[] lineEquation = findLineEquation(x1, y1, x2, y2);
System.out.println("Equation of the line: y = " + lineEquation[0] +
" * x + " + lineEquation[1]);

// Closing the scanner object
input.close();
}

// Method to find the Euclidean distance between two points
public static double findEuclideanDistance(double x1, double y1, double
x2, double y2) {
    // Calculating the distance using the formula
    double distance = Math.sqrt(Math.pow((x2 - x1), 2) + Math.pow((y2 -
y1), 2));
    return distance;
}

// Method to find the equation of a line given two points
public static double[] findLineEquation(double x1, double y1, double
x2, double y2) {
    // Calculating the slope (m)
    double m = (y2 - y1) / (x2 - x1);
    // Calculating the y-intercept (b)
    double b = y1 - (m * x1);
    // Returning an array with the slope and y-intercept
    return new double[]{m, b};
}
}
```

10. Write a program to find the 3 points that are collinear using the slope formulae and area of triangle formulae. check A (2, 4), B (4, 6) and C (6, 8) are Collinear for sampling.

Hint =>

a. Take inputs for 3 points x1, y1, x2, y2, and x3, y3

- b. Write a Method to find the 3 points that are collinear using the slope formula. The 3 points A(x1,y1), b(x2,y2), and c(x3,y3) are collinear if the slopes formed by 3 points ab, bc, and cd are equal.

$$\text{slope } AB = (y2 - y1)/(x2 - x1), \text{slope } BC = (y3 - y2)/(x3 - x2)$$

$$\text{slope } AC = (y3 - y1)/(x3 - x1) \text{ Points are collinear if}$$

$$\text{slope } AB = \text{slope } BC = \text{slope } AC$$

- c. The method to find the three points is collinear using the area of the triangle formula. The Three points are collinear if the area of the triangle formed by three points is 0. The area of a triangle is

$$\frac{1}{2} \begin{vmatrix} x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \end{vmatrix}$$

area of triangle formula

$$\frac{1}{2} \begin{vmatrix} 2 & 4 & 6 \\ 4 & 6 & 8 \end{vmatrix} = \frac{1}{2} \begin{vmatrix} -2 & -2 \\ -2 & -2 \end{vmatrix} = \frac{1}{2} (4 - 4) = 0$$

$$\text{area} = 0.5 * (x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2))$$

```
import java.util.Scanner;

public class CollinearPoints {
    // Method to check if 3 points are collinear using the slope formula
    public static boolean areCollinearUsingSlope(double x1, double y1,
double x2, double y2, double x3, double y3) {
        // Calculating the slopes
        double slopeAB = (y2 - y1) / (x2 - x1);
        double slopeBC = (y3 - y2) / (x3 - x2);
        double slopeAC = (y3 - y1) / (x3 - x1);
        // Checking if the slopes are
equal
        return (slopeAB == slopeBC)
(slopeBC == slopeAC);
    }

    // Method to check if 3 points are collinear using the area of the
triangle formula
    public static boolean areCollinearUsingArea(double x1, double y1,
double x2, double y2, double x3, double y3) {
```

$$\text{determinant} \begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc \quad \&\&$$

```
// Calculating the area of the triangle
double area = 0.5 * (x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 -
y2));
// Checking if the area is 0
return area == 0.0;
}

// Main method
public static void main(String[] args) {
    // Creating a scanner object to take user input
    Scanner input = new Scanner(System.in);

    // Taking user input for the coordinates of the three points
    System.out.print("Enter x1: ");
    double x1 = input.nextDouble();
    System.out.print("Enter y1: ");
    double y1 = input.nextDouble();
    System.out.print("Enter x2: ");
    double x2 = input.nextDouble();
    System.out.print("Enter y2: ");
    double y2 = input.nextDouble();
    System.out.print("Enter x3: ");
    double x3 = input.nextDouble();
    System.out.print("Enter y3: ");
    double y3 = input.nextDouble();

    // Checking if the points are collinear using the slope formula
    boolean collinearUsingSlope = areCollinearUsingSlope(x1, y1, x2,
y2, x3, y3);
    System.out.println("Collinear using slope formula: " +
collinearUsingSlope);

    // Checking if the points are collinear using the area of the
triangle formula
    boolean collinearUsingArea = areCollinearUsingArea(x1, y1, x2, y2,
x3, y3);
    System.out.println("Collinear using area of the triangle formula: "
+ collinearUsingArea);

    // Closing the scanner object
    input.close();
}
```

```
}  
}
```

11. Create a program to find the bonus of 10 employees based on their years of service as well as the total bonus amount the 10-year-old company Zara has to pay as a bonus, along with the old and new salary.

Hint =>

- Zara decides to give a bonus of 5% to employees whose year of service is more than 5 years or 2% if less than 5 years
- Create a Method to determine the Salary and years of service and return the same. Use the **Math.random()** method to determine the 5-digit salary for each employee and also use the random method to determine the years of service. Define 2D Array to save the salary and years of service.
- Write a Method to calculate the new salary and bonus based on the logic defined above and return the new 2D Array of the latest salary and bonus amount
- Write a Method to Calculate the sum of the Old Salary, the Sum of the New Salary, and the Total Bonus Amount and display it in a Tabular Format

```
public class EmployeeBonus {  
    // Method to generate random salary and years of service for employees  
    public static int[][] generateSalaryAndService(int numEmployees) {  
        // Creating 2D array to store salary and years of service  
        int[][] data = new int[numEmployees][2];  
        // Generating salary and years of service  
        for (int i = 0; i < numEmployees; i++) {  
            data[i][0] = (int)(Math.random() * 90000) + 10000; //  
Generating random 5-digit salary  
            data[i][1] = (int)(Math.random() * 11); // Generating random  
years of service (0-10)  
        }  
        return data;  
    }  
  
    // Method to calculate new salary and bonus  
    public static double[][] calculateNewSalaryAndBonus(int[][] data) {  
        int numEmployees = data.length;  
        // Creating 2D array to store new salary and bonus  
        double[][] newSalaryAndBonus = new double[numEmployees][2];  
        // Calculating new salary and bonus for each employee
```

```

    for (int i = 0; i < numEmployees; i++) {
        int salary = data[i][0];
        int yearsOfService = data[i][1];
        double bonusPercentage = yearsOfService > 5 ? 0.05 : 0.02;
        double bonus = salary * bonusPercentage;
        double newSalary = salary + bonus;
        // Saving new salary and bonus in the array
        newSalaryAndBonus[i][0] = newSalary;
        newSalaryAndBonus[i][1] = bonus;
    }
    return newSalaryAndBonus;
}

// Method to calculate and display the total bonus and salaries
public static void displayBonusAndSalaries(int[][] data, double[][]
newSalaryAndBonus) {
    // Creating variables to store total bonus and salaries
    int numEmployees = data.length;
    double totalOldSalary = 0;
    double totalNewSalary = 0;
    double totalBonus = 0;
    // Displaying table headers
    System.out.println("Employee Old Salary Years of Service New Salary
Bonus");
    for (int i = 0; i < numEmployees; i++) {
        // Fetching details
        int oldSalary = data[i][0];
        int yearsOfService = data[i][1];
        double newSalary = newSalaryAndBonus[i][0];
        double bonus = newSalaryAndBonus[i][1];
        // Displaying each employee's details
        System.out.println((i + 1) + " " + oldSalary + " " +
yearsOfService + " " + newSalary + " " + bonus);
        // Calculating total old salary, new salary and bonus
        totalOldSalary += oldSalary;
        totalNewSalary += newSalary;
        totalBonus += bonus;
    }
    // Displaying totals
    System.out.println("\nTotal " + totalOldSalary + " " +
totalNewSalary + " " + totalBonus);
}

```

```

    }

    // Main method
    public static void main(String[] args) {
        int numEmployees = 10;
        // Generating random salary and years of service for employees
        int[][] data = generateSalaryAndService(numEmployees);
        // Calculating new salary and bonus
        double[][] newSalaryAndBonus = calculateNewSalaryAndBonus(data);
        // Displaying bonus and salaries in tabular format
        displayBonusAndSalaries(data, newSalaryAndBonus);
    }
}

```

12. Create a program to take input marks of students in 3 subjects physics, chemistry, and maths. Compute the total, average, and the percentage score

Grade	Remarks	Marks
A	(Level 4, above agency-normalized standards)	80% and above
B	(Level 3, at agency-normalized standards)	70-79%
C	(Level 2, below, but approaching agency-normalized standards)	60-69%
D	(Level 1, well below agency-normalized standards)	50-59%
E	(Level 1- , too below agency-normalized standards)	40-49%
R	(Remedial standards)	39% and below

Hint =>

- Take input for the number of students
- Write a method to generate random 2-digit scores for Physics, Chemistry, and Math (PCM) for the students and return the scores. This method returns a 2D array with PCM scores for all students
- Write a Method to calculate the total, average, and percentages for each student and return a 2D array with the corresponding values. Please ensure to round off the values to 2 Digits using the ***Math.round()*** method.
- Finally, write a Method to display the scorecard of all students with their scores, total, average, and percentage in a tabular format using ***"\t"***.

```
import java.util.Scanner;
```

```

public class StudentScorecard {
    // Method to generate random 2-digit scores for PCM
    public static int[][] generateScores(int numStudents) {
        // Creating a 2D array to store scores
        int[][] scores = new int[numStudents][3];
        // Looping through each student to generate random scores of PCM
        for (int i = 0; i < numStudents; i++) {
            for (int j = 0; j < 3; j++) {
                scores[i][j] = (int)(Math.random() * 90) + 10; //
Generating random score between 10 and 99
            }
        }
        return scores;
    }

    // Method to calculate the total, average, and percentage for each
student
    public static double[][] calculateResults(int[][] scores) {
        // Calculating the size of score array
        int numStudents = scores.length;
        // Creating a 2D array to store results
        double[][] results = new double[numStudents][3];
        // Looping through each student to calculate total, average, and
percentage
        for (int i = 0; i < numStudents; i++) {
            int total = scores[i][0] + scores[i][1] + scores[i][2];
            double average = total / 3.0;
            double percentage = (total / 300.0) * 100;
            results[i][0] = total;
            results[i][1] = Math.round(average * 100.0) / 100.0;
            results[i][2] = Math.round(percentage * 100.0) / 100.0;
        }
        return results;
    }

    // Method to display the scorecard in a tabular format
    public static void displayScorecard(int[][] scores, double[][] results)
{
    System.out.println("Student\tPhysics\tChemistry\tMaths\tTotal\tAverage\tPer

```

```
centage");
    // Looping through each student to display the scorecard
    for (int i = 0; i < scores.length; i++) {
        // Displaying student number
        System.out.print((i + 1) + "\t");
        for (int j = 0; j < 3; j++) {
            // Displaying student marks of PCM
            System.out.print(scores[i][j] + "\t");
        }
        // Displaying student results of total, average, and percentage
        System.out.print(results[i][0] + "\t");
        System.out.print(results[i][1] + "\t");
        System.out.println(results[i][2]);
    }
}

// Main method
public static void main(String[] args) {
    // Creating a scanner object to take user input
    Scanner input = new Scanner(System.in);
    // Taking input for the number of students
    System.out.print("Enter the number of students: ");
    int numStudents = input.nextInt();
    // Generating random scores for PCM
    int[][] scores = generateScores(numStudents);
    // Calculating total, average, and percentage
    double[][] results = calculateResults(scores);
    // Displaying the scorecard
    displayScorecard(scores, results);
    // Closing the Scanner object
    input.close();
}
}
```

13. Write a program to perform matrix manipulation operations like addition, subtraction, multiplication, and transpose. Also finding the determinant and inverse of a matrix. The program should take random matrices as input and display the result of the operations.

Hint =>

- Write a Method to create a random matrix taking rows and columns as parameters
- Write a Method to add two matrices

- c. Write a Method to subtract two matrices
- d. Write a Method to multiply two matrices

Rectangular matrices [\[edit\]](#)

If

$$\mathbf{A} = \begin{pmatrix} a & b & c \\ x & y & z \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} \alpha & \rho \\ \beta & \sigma \\ \gamma & \tau \end{pmatrix},$$

their matrix products are:

$$\mathbf{AB} = \begin{pmatrix} a & b & c \\ x & y & z \end{pmatrix} \begin{pmatrix} \alpha & \rho \\ \beta & \sigma \\ \gamma & \tau \end{pmatrix} = \begin{pmatrix} a\alpha + b\beta + c\gamma & a\rho + b\sigma + c\tau \\ x\alpha + y\beta + z\gamma & x\rho + y\sigma + z\tau \end{pmatrix},$$

and

$$\mathbf{BA} = \begin{pmatrix} \alpha & \rho \\ \beta & \sigma \\ \gamma & \tau \end{pmatrix} \begin{pmatrix} a & b & c \\ x & y & z \end{pmatrix} = \begin{pmatrix} \alpha a + \rho x & \alpha b + \rho y & \alpha c + \rho z \\ \beta a + \sigma x & \beta b + \sigma y & \beta c + \sigma z \\ \gamma a + \tau x & \gamma b + \tau y & \gamma c + \tau z \end{pmatrix}.$$

- e. Write a Method to find the transpose of a matrix

$$\mathbf{M} = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{bmatrix} \quad \mathbf{M}^T = \begin{bmatrix} 1 & 0 & 5 \\ 2 & 1 & 6 \\ 3 & 4 & 0 \end{bmatrix}$$

- f. Write a Method to find the determinant of a 2x2 matrix
- g. Write a Method to find the determinant of a 3x3 matrix

$$\mathbf{M} = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{bmatrix}$$

$$\det(\mathbf{M}) = 1(0-24) - 2(0-20) + 3(0-5) = 1$$

$$\mathbf{M}^T = \begin{bmatrix} 1 & 0 & 5 \\ 2 & 1 & 6 \\ 3 & 4 & 0 \end{bmatrix}$$

$\begin{vmatrix} 1 & 6 \\ 4 & 0 \end{vmatrix} = -24$	$\begin{vmatrix} 2 & 6 \\ 3 & 0 \end{vmatrix} = -18$	$\begin{vmatrix} 2 & 1 \\ 3 & 4 \end{vmatrix} = 5$
$\begin{vmatrix} 0 & 5 \\ 4 & 0 \end{vmatrix} = -20$	$\begin{vmatrix} 1 & 5 \\ 3 & 0 \end{vmatrix} = -15$	$\begin{vmatrix} 1 & 0 \\ 3 & 4 \end{vmatrix} = 4$
$\begin{vmatrix} 0 & 5 \\ 1 & 6 \end{vmatrix} = -5$	$\begin{vmatrix} 1 & 5 \\ 2 & 6 \end{vmatrix} = -4$	$\begin{vmatrix} 1 & 0 \\ 2 & 1 \end{vmatrix} = 1$

- h. Write a Method to find the inverse of a 2x2 matrix
- i. Write a Method to find the inverse of a 3x3 matrix
- j. Write a Method to display a matrix

```
import java.util.Scanner;

public class SimpleMatrixOperations {

    // Method to create a random matrix
```

```

public static int[][] createRandomMatrix(int rows, int cols) {
    // Create a 2D matrix
    int[][] matrix = new int[rows][cols];
    // Iterating through loop to generate and save random numbers to
matrix
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i][j] = (int) (Math.random() * 10); // Random value
between 0 and 9
        }
    }
    return matrix;
}

// Method to add two matrices
public static int[][] addMatrices(int[][] matrix1, int[][] matrix2) {
    // Finding the size of the matrices
    int rows = matrix1.length;
    int cols = matrix1[0].length;
    // Create a new matrix with the same size as the input matrices
    int[][] result = new int[rows][cols];
    // Iterating through loop to add elements of the matrices
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            result[i][j] = matrix1[i][j] + matrix2[i][j];
        }
    }
    return result;
}

// Method to subtract two matrices
public static int[][] subtractMatrices(int[][] matrix1, int[][]
matrix2) {
    // Finding the size of the matrices
    int rows = matrix1.length;
    int cols = matrix1[0].length;
    // Create a new matrix with the same size as the input matrices
    int[][] result = new int[rows][cols];
    // Iterating through loop to subtract elements of the matrices
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {

```

```

        result[i][j] = matrix1[i][j] - matrix2[i][j];
    }
}
return result;
}

// Method to multiply two matrices
public static int[][] multiplyMatrices(int[][] matrix1, int[][]
matrix2) {
    // Finding the size of the matrices
    int rows1 = matrix1.length;
    int cols1 = matrix1[0].length;
    int cols2 = matrix2[0].length;
    // Create a new matrix with the same size as the input matrices
    int[][] result = new int[rows1][cols2];
    // Iterating through loop to multiply elements of the matrices
    for (int i = 0; i < rows1; i++) {
        for (int j = 0; j < cols2; j++) {
            result[i][j] = 0;
            for (int k = 0; k < cols1; k++) {
                result[i][j] += matrix1[i][k] * matrix2[k][j];
            }
        }
    }
    return result;
}

// Method to find the transpose of a matrix
public static int[][] transposeMatrix(int[][] matrix) {
    // Finding the size of the matrix
    int rows = matrix.length;
    int cols = matrix[0].length;
    // Create a new matrix with the same size as the input matrix
    int[][] result = new int[cols][rows];
    // Iterating through loop to find the transpose of the matrix
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            result[j][i] = matrix[i][j];
        }
    }
    return result;
}

```

```

}

// Method to find the determinant of a 2x2 matrix
public static int determinant2x2(int[][] matrix) {
    // Calculating the determinant of the 2x2 matrix and return
    return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0];
}

// Method to find the determinant of a 3x3 matrix
public static int determinant3x3(int[][] matrix) {
    // Calculating the determinant of the 3x3 matrix
    int det = matrix[0][0] * (matrix[1][1] * matrix[2][2] -
matrix[1][2] * matrix[2][1])
        - matrix[0][1] * (matrix[1][0] * matrix[2][2] -
matrix[1][2] * matrix[2][0])
        + matrix[0][2] * (matrix[1][0] * matrix[2][1] -
matrix[1][1] * matrix[2][0]);
    return det;
}

// Method to find the inverse of a 2x2 matrix
public static double[][] inverse2x2(int[][] matrix) {
    // Calculating the determinant of the 2x2 matrix
    int det = determinant2x2(matrix);
    // checking if the determinant is zero
    if (det == 0) {
        throw new IllegalArgumentException("Matrix is singular and
cannot be inverted.");
    }
    // Create an inverse array
    double[][] inverse = new double[2][2];
    // Calculating the inverse of the 2x2 matrix
    inverse[0][0] = matrix[1][1] / (double) det;
    inverse[0][1] = -matrix[0][1] / (double) det;
    inverse[1][0] = -matrix[1][0] / (double) det;
    inverse[1][1] = matrix[0][0] / (double) det;
    return inverse;
}

// Method to find the inverse of a 3x3 matrix
public static double[][] inverse3x3(int[][] matrix) {

```

```
// Calculating the determinant of the 3x3 matrix
int det = determinant3x3(matrix);
// checking if the determinant is zero
if (det == 0) {
    throw new IllegalArgumentException("Matrix is singular and
cannot be inverted.");
}
// Create an inverse array
double[][] inverse = new double[3][3];
// Calculating the inverse of the 3x3 matrix
inverse[0][0] = (matrix[1][1] * matrix[2][2] - matrix[1][2] *
matrix[2][1]) / (double) det;
inverse[0][1] = (matrix[0][2] * matrix[2][1] - matrix[0][1] *
matrix[2][2]) / (double) det;
inverse[0][2] = (matrix[0][1] * matrix[1][2] - matrix[0][2] *
matrix[1][1]) / (double) det;
inverse[1][0] = (matrix[1][2] * matrix[2][0] - matrix[1][0] *
matrix[2][2]) / (double) det;
inverse[1][1] = (matrix[0][0] * matrix[2][2] - matrix[0][2] *
matrix[2][0]) / (double) det;
inverse[1][2] = (matrix[0][2] * matrix[1][0] - matrix[0][0] *
matrix[1][2]) / (double) det;
inverse[2][0] = (matrix[1][0] * matrix[2][1] - matrix[1][1] *
matrix[2][0]) / (double) det;
inverse[2][1] = (matrix[0][1] * matrix[2][0] - matrix[0][0] *
matrix[2][1]) / (double) det;
inverse[2][2] = (matrix[0][0] * matrix[1][1] - matrix[0][1] *
matrix[1][0]) / (double) det;
return inverse;
}

// Method to display a matrix
public static void displayMatrix(int[][] matrix) {
    // Iterating through loop to display matrix
    for (int i = 0; i < matrix.length; i++) {
        for (int j = 0; j < matrix[i].length; j++) {
            System.out.print(matrix[i][j] + "\t");
        }
        // To start from a new row
        System.out.println();
    }
}
```

```

}

// Method to display a double matrix
public static void displayDoubleMatrix(double[][] matrix) {
    // Iterating through loop to display matrix
    for (int i = 0; i < matrix.length; i++) {
        for (int j = 0; j < matrix[i].length; j++) {
            System.out.print(matrix[i][j] + "\t");
        }
        // To start from a new row
        System.out.println();
    }
}

// Main method
public static void main(String[] args) {
    // Create a scanner object to take user input
    Scanner input = new Scanner(System.in);

    // Take input for the size of matrices
    System.out.print("Enter the number of rows for the matrices: ");
    int rows = input.nextInt();
    System.out.print("Enter the number of columns for the matrices: ");
    int cols = input.nextInt();

    // Generate two random matrices
    int[][] matrix1 = createRandomMatrix(rows, cols);
    int[][] matrix2 = createRandomMatrix(rows, cols);

    // Perform and display matrix operations
    System.out.println("Matrix 1:");
    displayMatrix(matrix1);
    System.out.println("Matrix 2:");
    displayMatrix(matrix2);

    // Addition
    System.out.println("Addition of Matrix 1 and Matrix 2:");
    displayMatrix(addMatrices(matrix1, matrix2));

    // Subtraction
    System.out.println("Subtraction of Matrix 1 and Matrix 2:");
}

```

```

displayMatrix(subtractMatrices(matrix1, matrix2));

// Multiplication (if applicable)
if (cols == matrix2.length) {
    System.out.println("Multiplication of Matrix 1 and Matrix 2:");
    displayMatrix(multiplyMatrices(matrix1, matrix2));
} else {
    System.out.println("Multiplication not possible due to
incompatible matrix dimensions.");
}

// Transpose
System.out.println("Transpose of Matrix 1:");
displayMatrix(transposeMatrix(matrix1));

if (rows == 2 && cols == 2) {
    // Determinant and inverse for 2x2 matrix
    System.out.println("Determinant of Matrix 1:");
    System.out.println(determinant2x2(matrix1));
    System.out.println("Inverse of Matrix 1:");
    displayDoubleMatrix(inverse2x2(matrix1));
} else if (rows == 3 && cols == 3) {
    // Determinant and inverse for 3x3 matrix
    System.out.println("Determinant of Matrix 1:");
    System.out.println(determinant3x3(matrix1));
    System.out.println("Inverse of Matrix 1:");
    displayDoubleMatrix(inverse3x3(matrix1));
} else {
    System.out.println("Determinant and inverse not supported for
this matrix size.");
}

// Close the Scanner object
input.close();
}
}

```